

Ans

April 10, 2024

```
[167]: import pandas as pd
import sklearn
from sklearn import metrics
```

```
[168]: df = pd.read_csv('Diabetes.csv')
df.head()
```

```
[168]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

```
[169]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Pregnancies                          768 non-null    int64
1   Glucose                              768 non-null    int64
2   BloodPressure                        768 non-null    int64
3   SkinThickness                        768 non-null    int64
4   Insulin                              768 non-null    int64
5   BMI                                  768 non-null    float64
6   DiabetesPedigreeFunction              768 non-null    float64
7   Age                                  768 non-null    int64
8   Outcome                              768 non-null    int64
```

```
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
[170]: df.describe()
```

```
[170]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479
std	3.369578	31.972618	19.355807	15.952218	115.244002
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

```
[171]: df.isnull().sum()
```

```
[171]: Pregnancies      0
Glucose              0
BloodPressure        0
SkinThickness        0
Insulin              0
BMI                  0
DiabetesPedigreeFunction  0
Age                  0
Outcome              0
dtype: int64
```

```
[172]: # Doesn't make sense for any column except pregnancy or outcome to be zero.
# Replace 0s with nulls in those cases.
zero_columns = [col for col in df.columns if (df[col].min() == 0 and col not in
↳ ["Pregnancies", "Outcome"])]
for col in zero_columns:
    df[col] = np.where(df[col]==0, np.nan, df[col])
```

```
[173]: df.isnull().sum()
```

```
[173]: Pregnancies      0
      Glucose          5
      BloodPressure    35
      SkinThickness    227
      Insulin          374
      BMI              11
      DiabetesPedigreeFunction  0
      Age              0
      Outcome          0
      dtype: int64
```

```
[174]: mean = df.mean()

      df.fillna(mean, inplace=True)
```

```
[189]: df.describe()
```

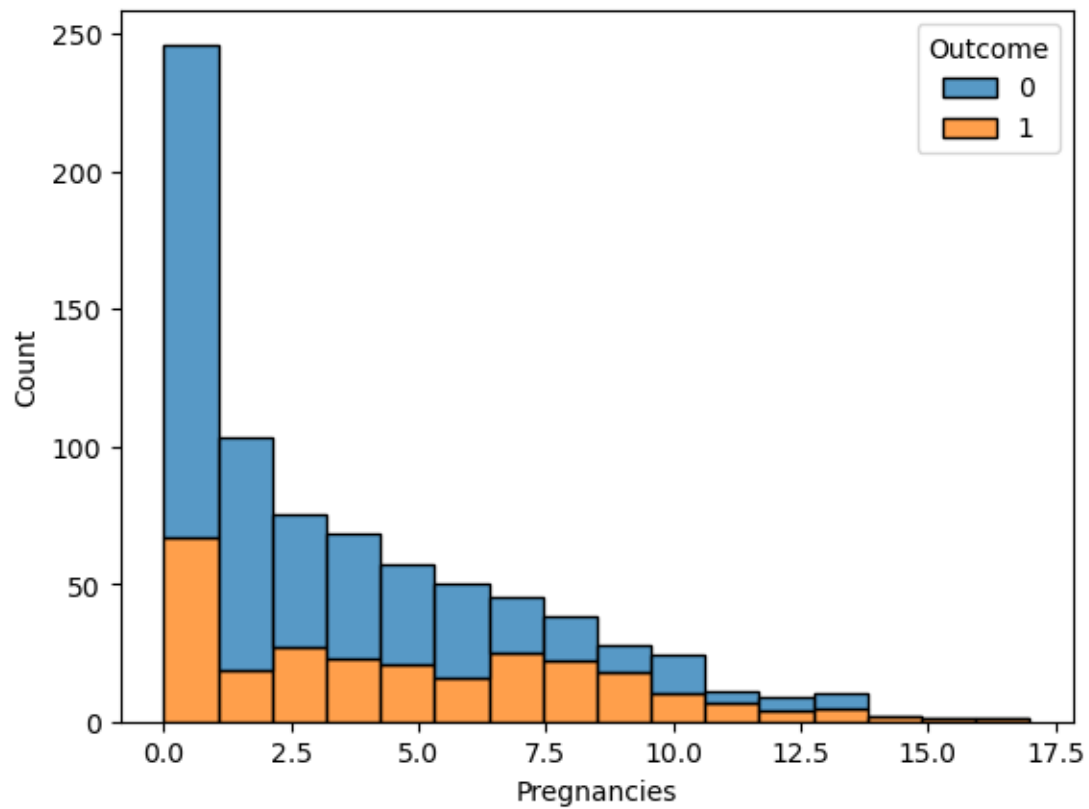
```
[189]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.686763	72.405184	29.153420	155.548223
std	3.369578	30.435949	12.096346	8.790942	85.021108
min	0.000000	44.000000	24.000000	7.000000	14.000000
25%	1.000000	99.750000	64.000000	25.000000	121.500000
50%	3.000000	117.000000	72.202592	29.153420	155.548223
75%	6.000000	140.250000	80.000000	32.000000	155.548223
max	17.000000	199.000000	122.000000	99.000000	846.000000

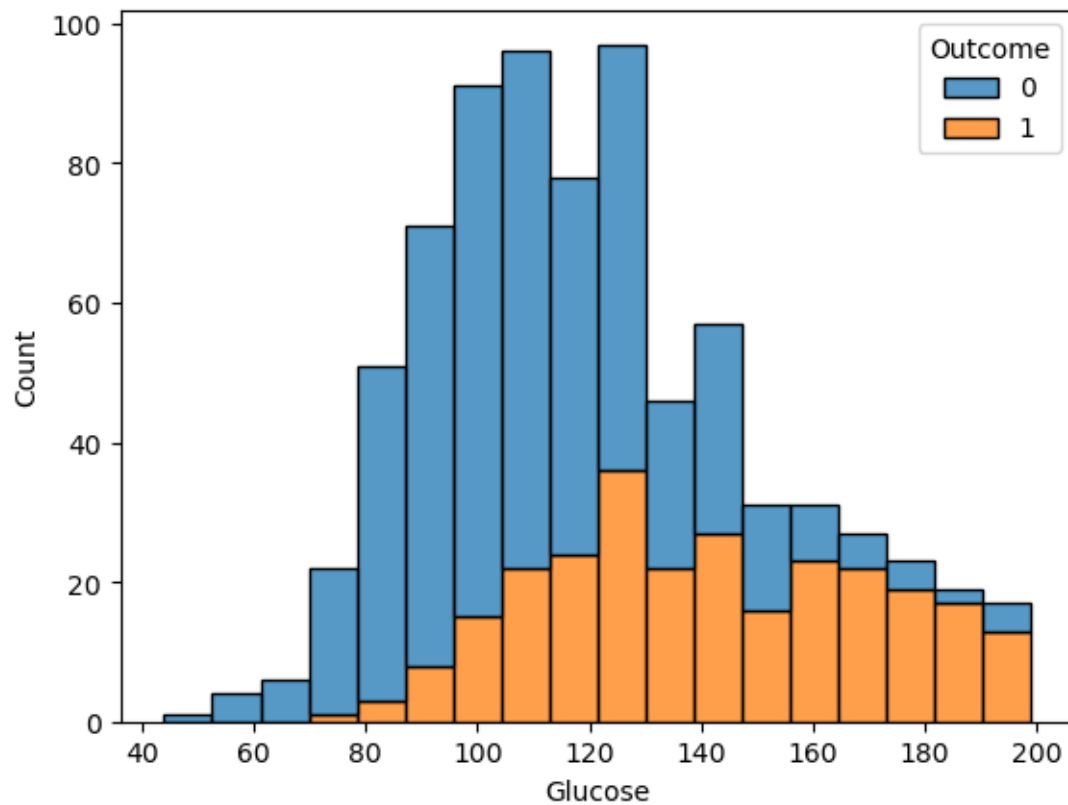
	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	32.457464	0.471876	33.240885	0.348958
std	6.875151	0.331329	11.760232	0.476951
min	18.200000	0.078000	21.000000	0.000000
25%	27.500000	0.243750	24.000000	0.000000
50%	32.400000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

```
[175]: import seaborn as sns
      import matplotlib.pyplot as plt
      for col in df.columns[:-1]:
          sns.histplot(df, x=col, hue="Outcome", multiple='stack')
          plt.show()
```

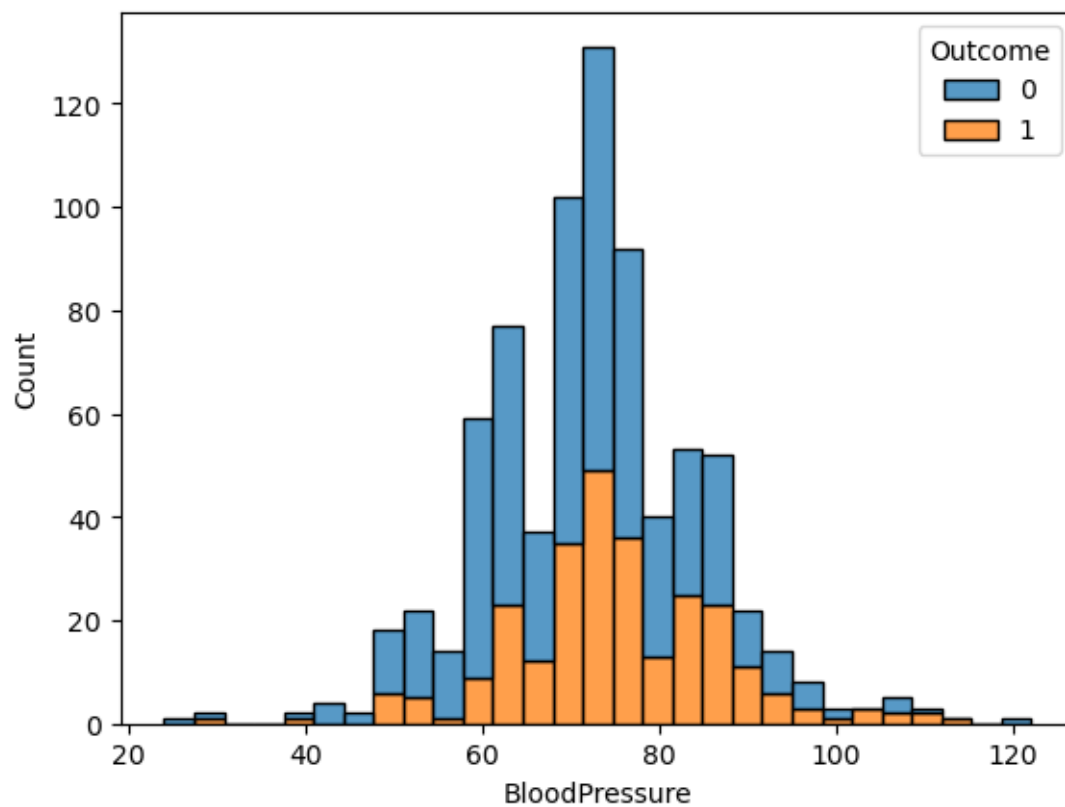
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
      with pd.option_context('mode.use_inf_as_na', True):
```



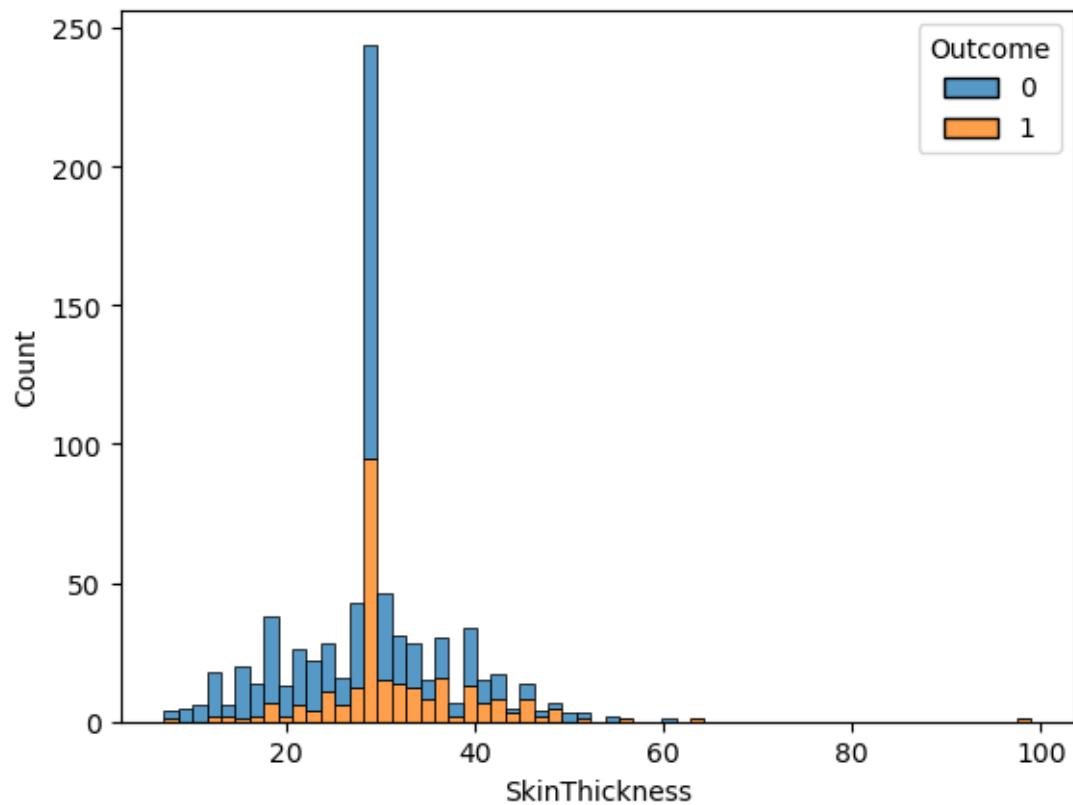
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



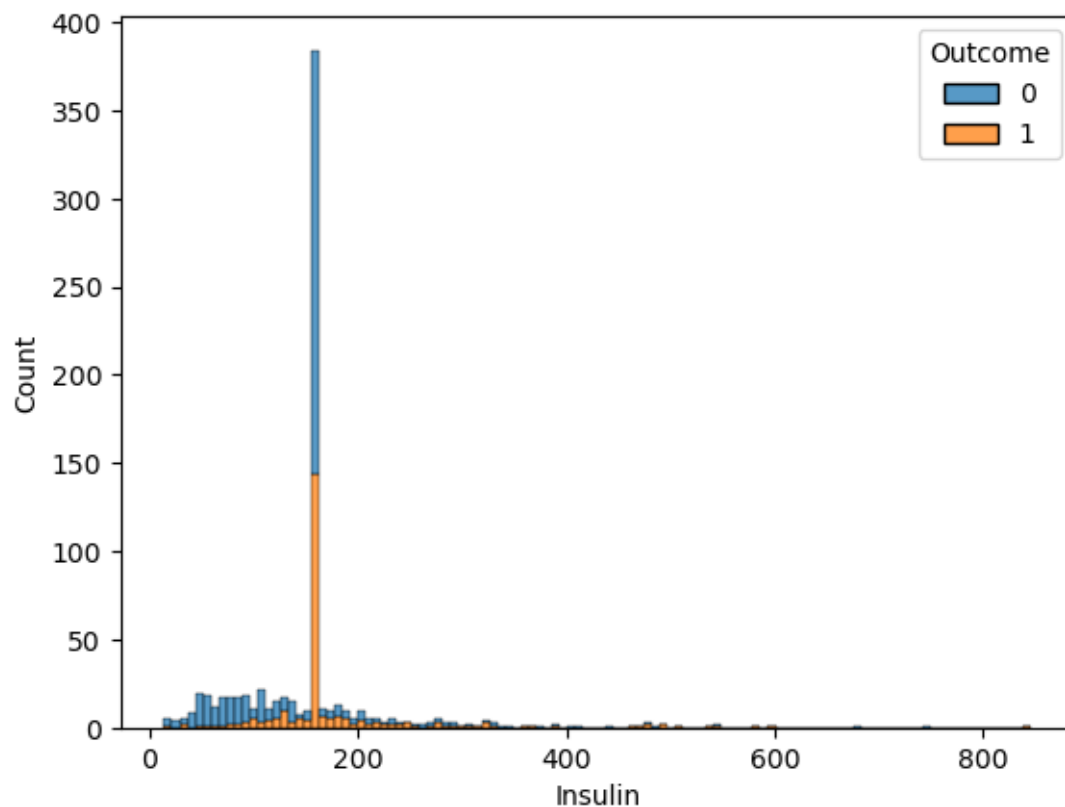
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



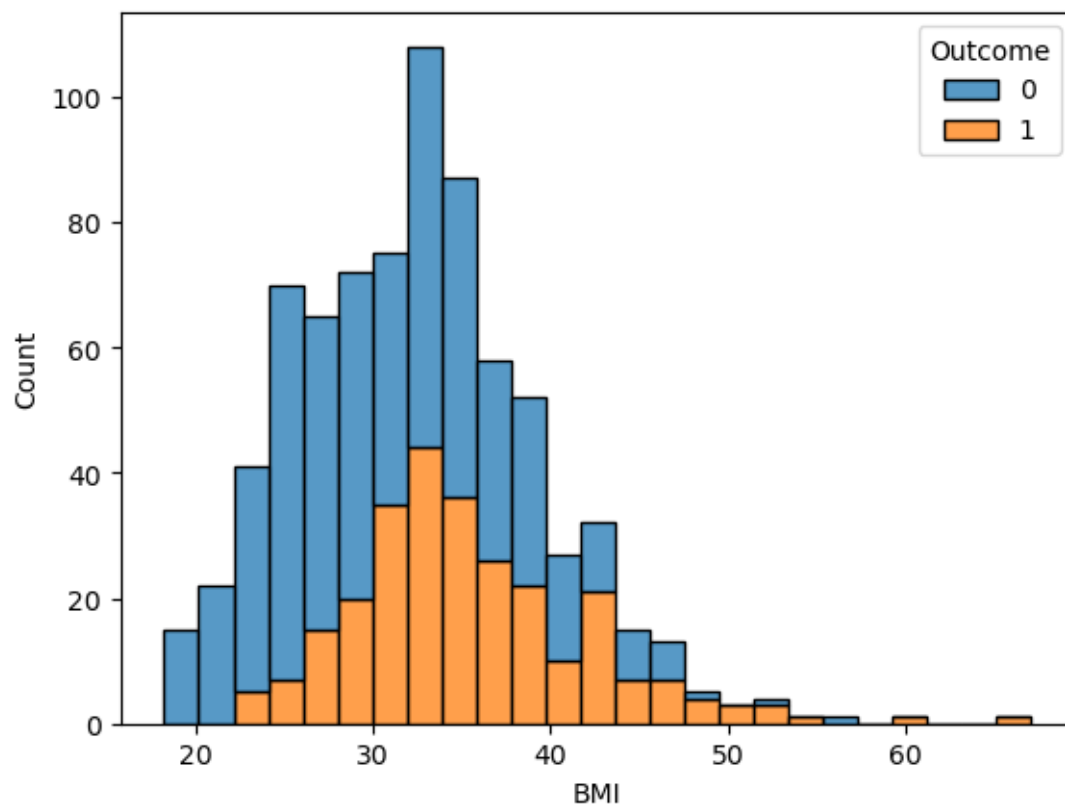
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



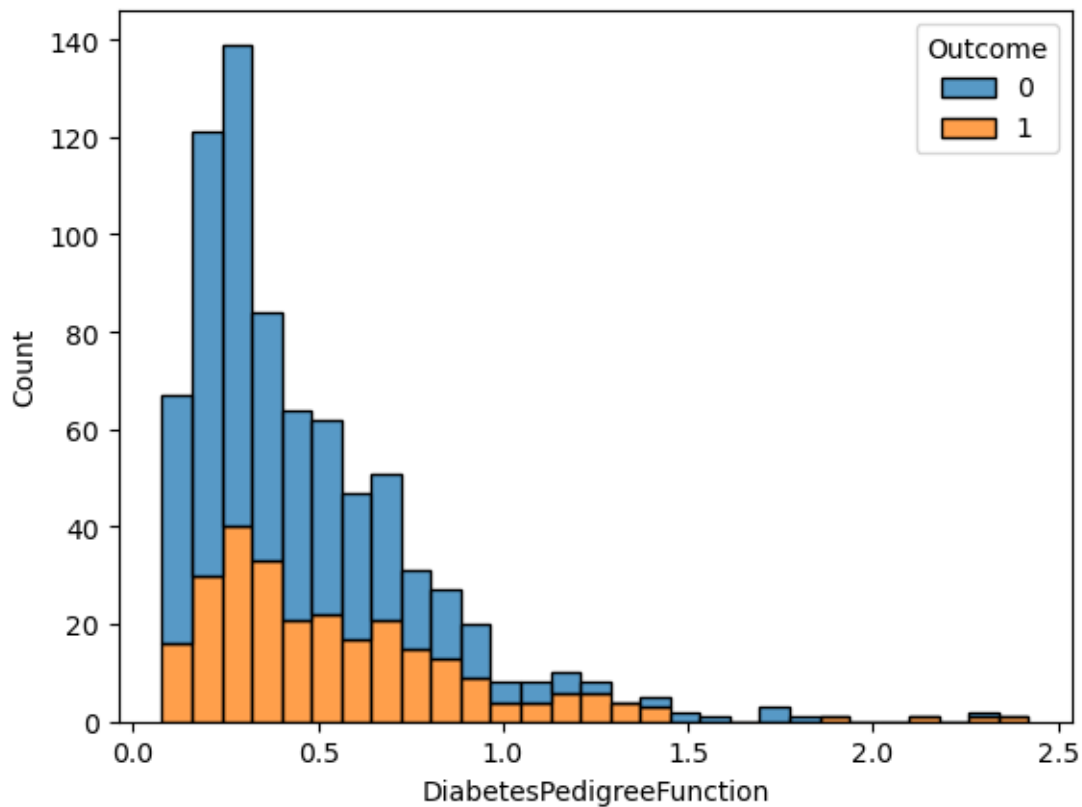
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



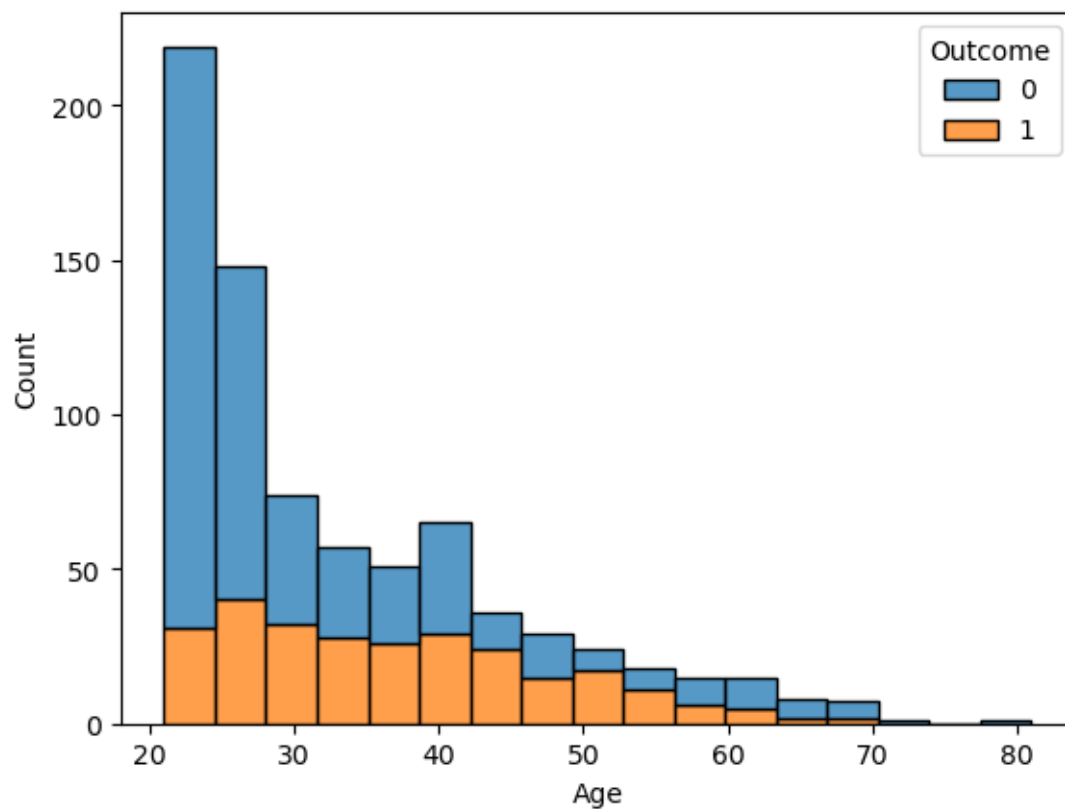
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



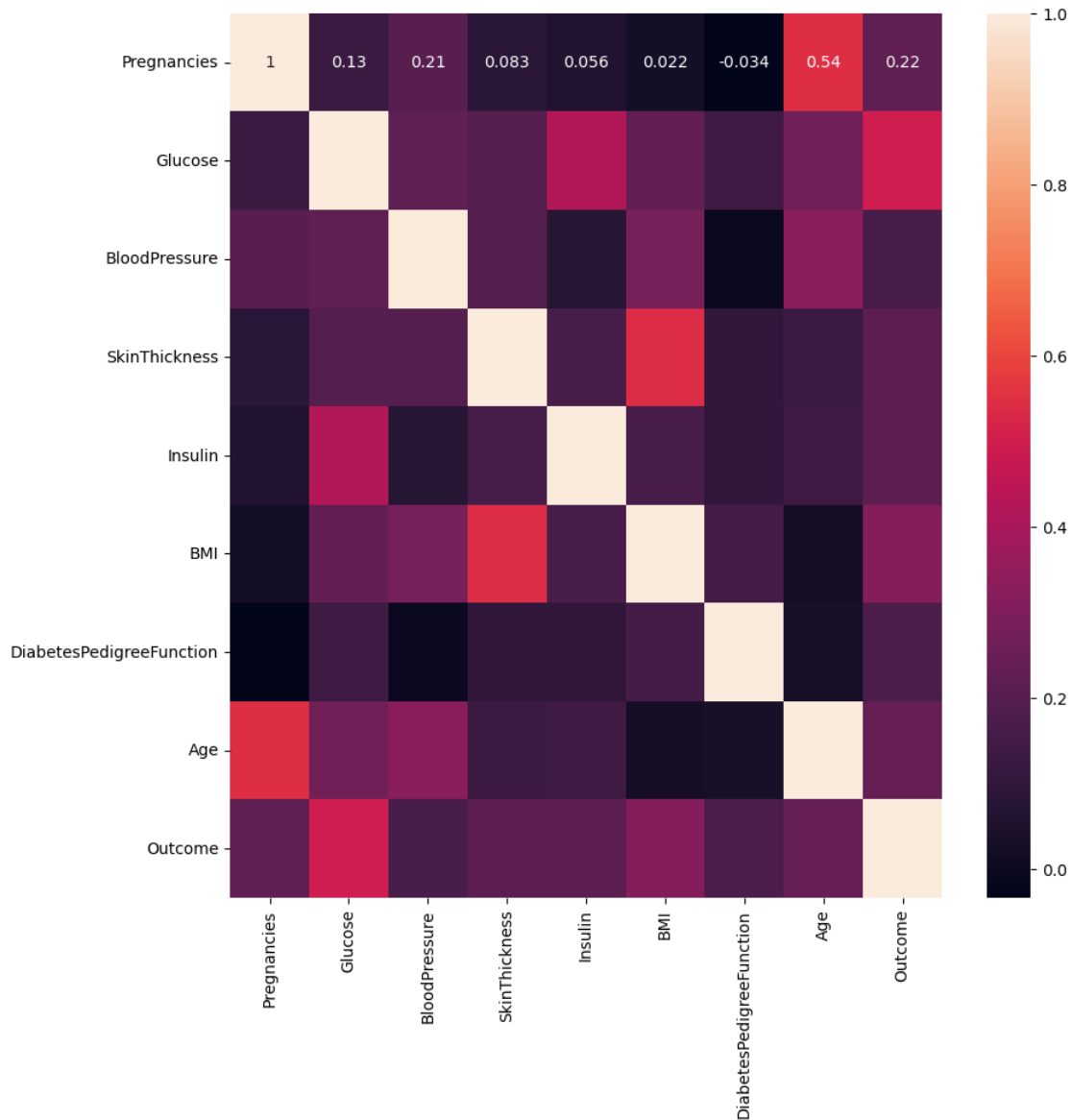
```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



```
[176]: plt.figure(figsize=(10,10))
sns.heatmap(df.corr(),annot=True)
plt.show()
```



```
[177]: from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np

np.random.seed(42)
X = df.drop(['Outcome'],axis=1)
y = df["Outcome"]

# Split the dataset into training and testing sets
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

# Create a Perceptron model
perceptron_model = Perceptron(max_iter=20, eta0=0.01, random_state=42)

# Train the model
perceptron_model.fit(X_train, y_train)

```

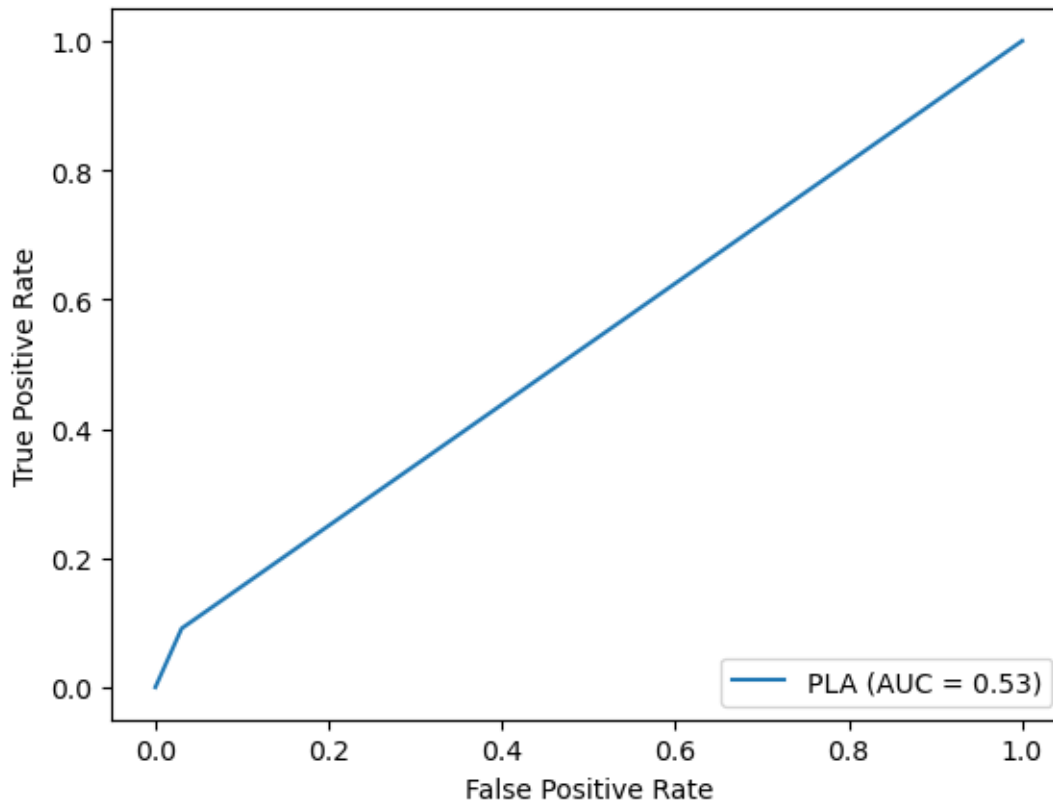
[177]: Perceptron(eta0=0.01, max_iter=20, random_state=42)

```

[178]: import matplotlib.pyplot as plt
# Make predictions on the test set
y_pred = perceptron_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(fpr=fpr, tpr=tpr,
↳roc_auc=roc_auc, estimator_name='PLA')
display.plot()
plt.show()
print(f"Accuracy on the test set With PLA: {accuracy}")

```



Accuracy on the test set With PLA: 0.6558441558441559

```
[179]: from sklearn.neural_network import MLPClassifier
# Create an MLPClassifier model
mlp_model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=100, alpha=0.01,
    random_state=42)

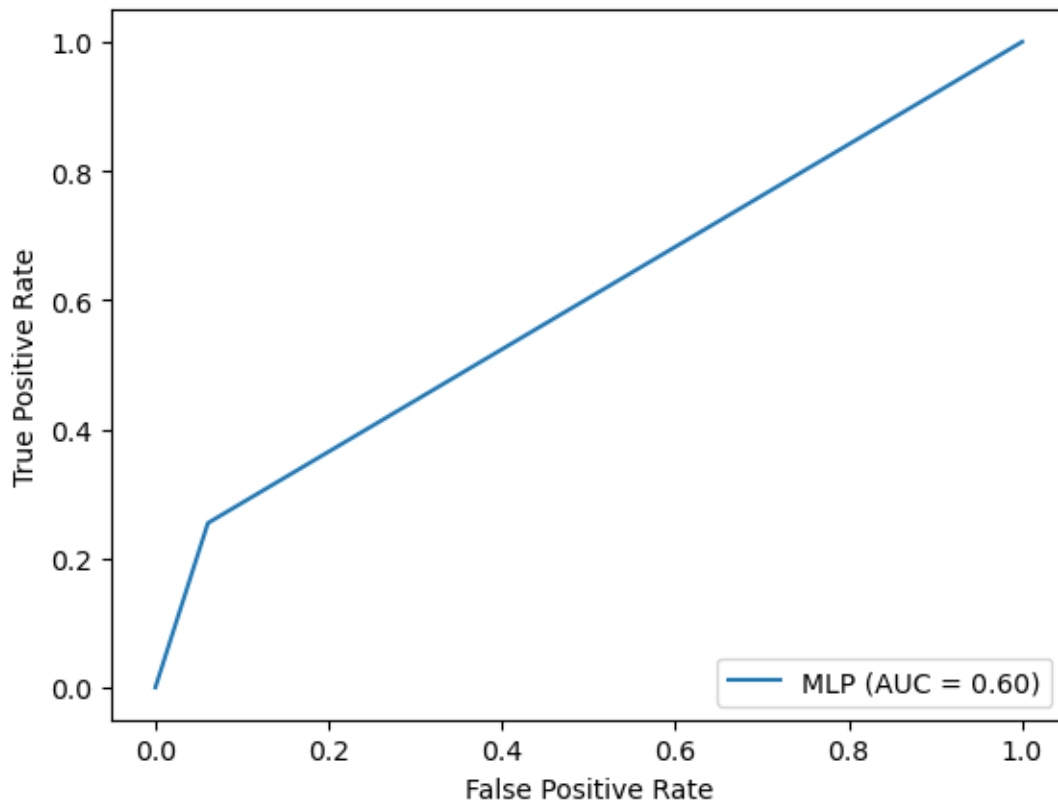
# Train the model
mlp_model.fit(X_train, y_train)
```

```
[179]: MLPClassifier(alpha=0.01, max_iter=100, random_state=42)
```

```
[180]: # Make predictions on the test set
y_pred = mlp_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(fpr=fpr, tpr=tpr,
    roc_auc=roc_auc, estimator_name='MLP')
```

```
display.plot()
plt.show()
print(f"Accuracy on the test set (MLP): {accuracy}")
```

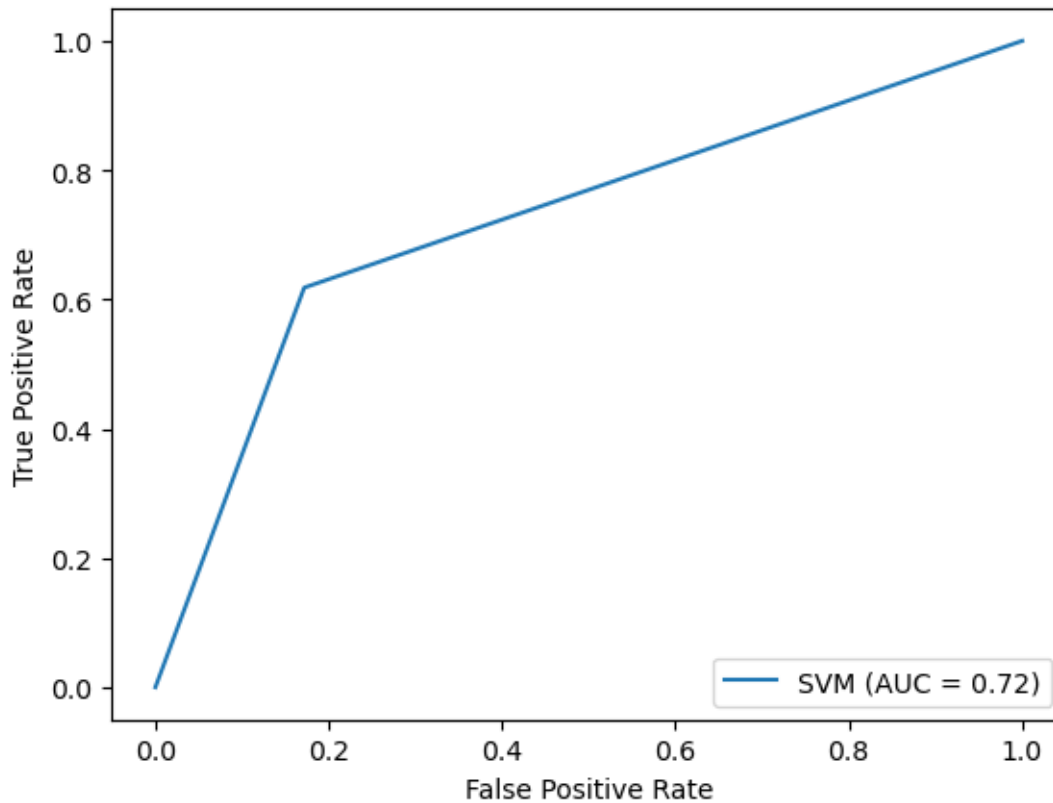


Accuracy on the test set (MLP): 0.6948051948051948

```
[181]: from sklearn import svm
svmmodel = svm.SVC(kernel='linear')
svmmodel.fit(X_train, y_train)
```

```
[181]: SVC(kernel='linear')
```

```
[182]: y_pred = svmmodel.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(fpr=fpr, tpr=tpr,
    ↳roc_auc=roc_auc, estimator_name='SVM')
display.plot()
plt.show()
print("Accuracy of SVM:", accuracy)
```



Accuracy of SVM: 0.7532467532467533

```
[183]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

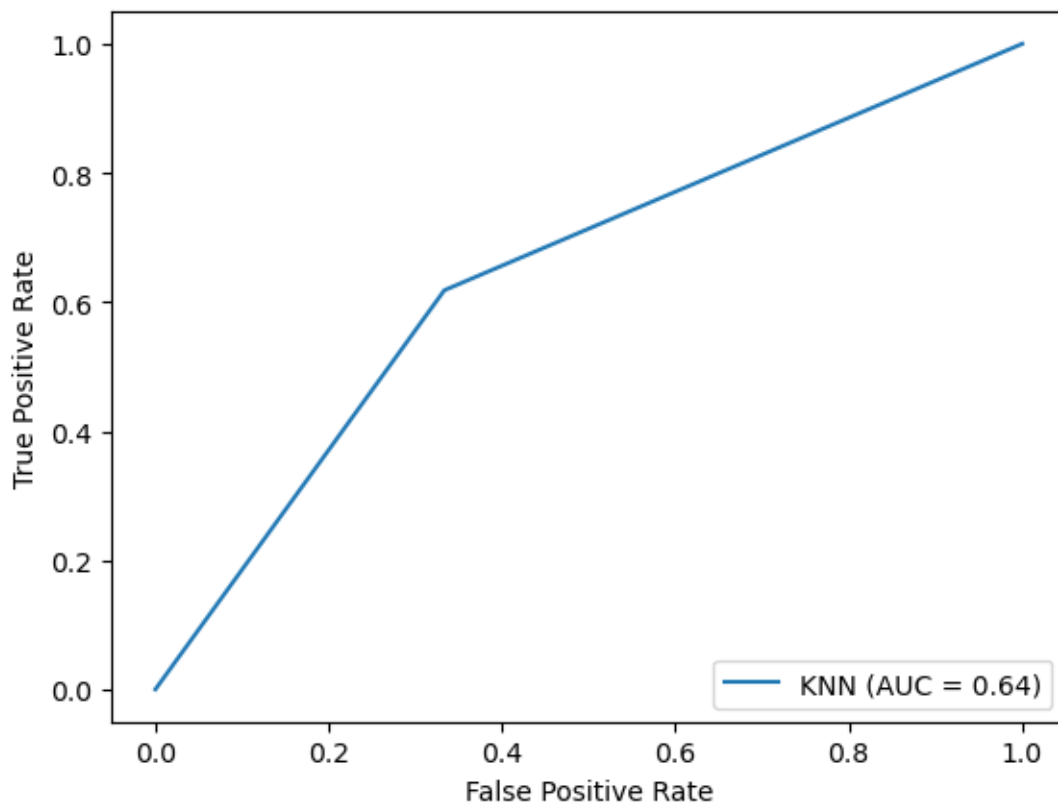
# Initialize the KNN classifier
knnmodel = KNeighborsClassifier(n_neighbors=5) # You can adjust the number of
↪neighbors

# Train the model
knnmodel.fit(X_train, y_train)
```

```
[183]: KNeighborsClassifier()
```

```
[184]: y_pred = knnmodel.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(fpr=fpr, tpr=tpr,
↪roc_auc=roc_auc, estimator_name='KNN')
display.plot()
```

```
plt.show()
print("Accuracy of KNN:", accuracy)
```



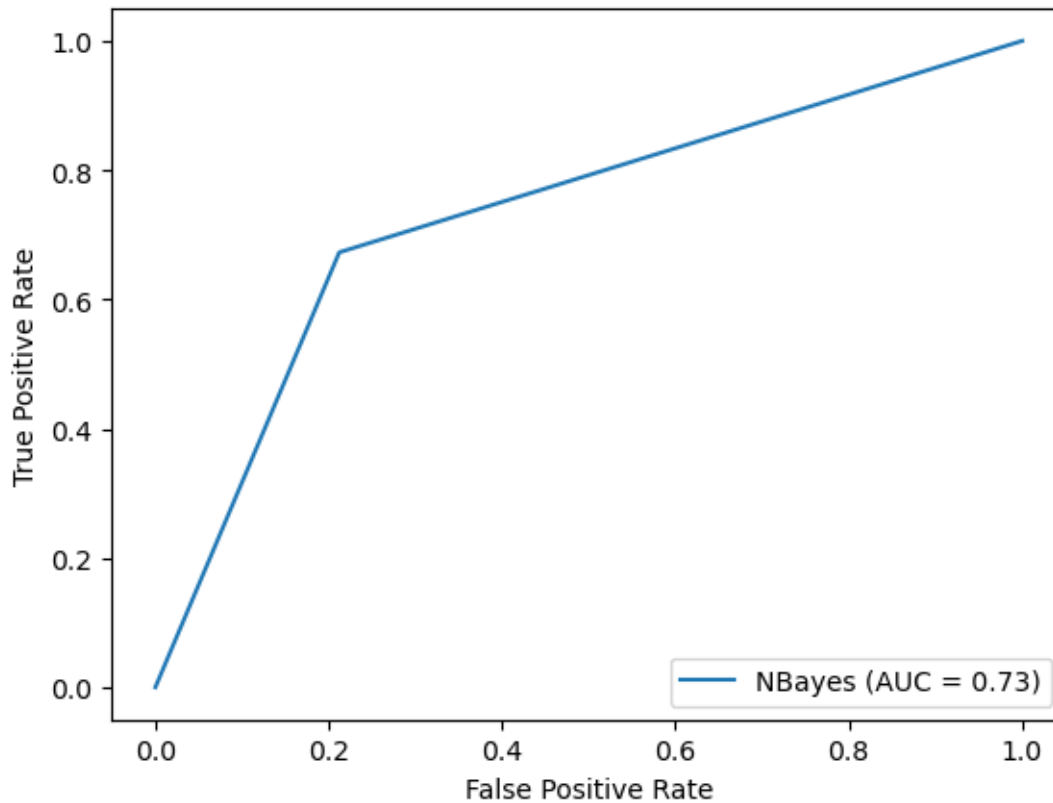
Accuracy of KNN: 0.6493506493506493

```
[185]: from sklearn.naive_bayes import GaussianNB
```

```
nbmodel = GaussianNB()
nbmodel.fit(X_train,y_train)
```

```
[185]: GaussianNB()
```

```
[186]: y_pred = nbmodel.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(fpr=fpr, tpr=tpr,
    ↳roc_auc=roc_auc,estimator_name='NBayes')
display.plot()
plt.show()
print("Accuracy of NBayes:", accuracy)
```



Accuracy of NBayes: 0.7467532467532467

```
[187]: from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=2, random_state=42) # Adjust the number of clusters
        ↪ as needed

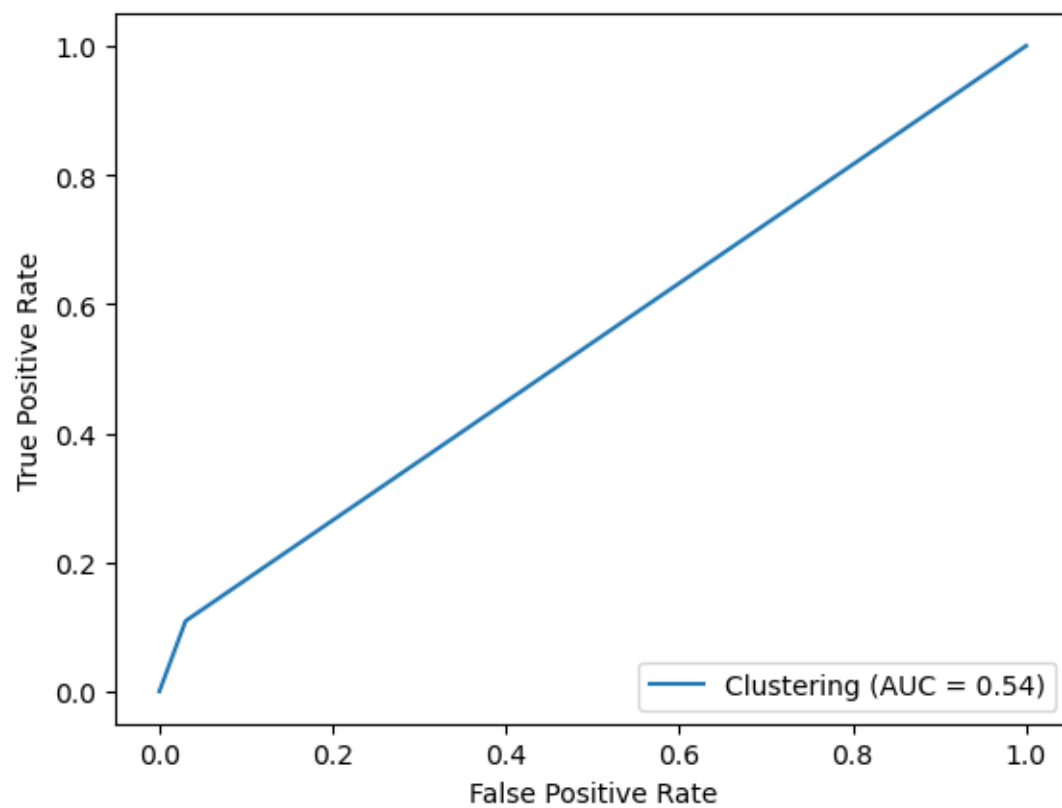
y_train_pred = kmeans.fit(X_train)
y_pred = kmeans.predict(X_test)
accuracy_score(y_test, y_pred)
```

```
/opt/anaconda3/lib/python3.11/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
```

[187]: 0.6623376623376623

```
[188]: fpr, tpr, thresholds = metrics.roc_curve(y_test, y_pred)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(fpr=fpr, tpr=tpr,
        ↪ roc_auc=roc_auc, estimator_name='Clustering')
```

```
display.plot()  
plt.show()
```



```
[ ]:
```